Validation Placement in Static Multi-UAV Language Planning: A Paired Failure-Containment and Compute Study

Anonymous Author(s)
Affiliation
Address
email

Abstract

Natural-language planners for multi-robot systems can produce well-formed plans even when required evidence is missing or available resources conflict with the requested mission. This paper examines whether four validation-pipeline configurations differ in their ability to contain such cases and in their local inference costs. From 284 held-out MultiUAV-Plat source tasks, we construct five linked variants per task, yielding 1,420 cases per method for each of two frozen Qwen2.5 checkpoints. The compared configurations are monolithic planning (M1), deterministic post-plan gating (M2), stage-wise evidence and provenance validation (M3), and a two-call post-plan configuration matched to M3 only by model-call count (M4). With Qwen2.5-7B, M3 produced no unsupported continuation in 568 registered non-executable cases, compared with 388 cases for M1 and 15 for M4. Its registered all-case success rate was 228/1,420, but all 228 successes were correct containment outcomes; every method had zero static plan fidelity on the 852 executable cases. With Qwen2.5-3B, M3 improved containment relative to M1, yet all four methods had zero registered all-case success. A separate three-repetition RTX 3090 campaign showed model-dependent compute trade-offs: relative to M1, M3 increased time and GPU-board energy for the 3B checkpoint but reduced both for the 7B checkpoint. These findings show that pipeline configuration can change static failure containment and local resource use. They do not establish executable plan fidelity, physical-flight safety, or generality beyond the tested benchmark, models, and hardware.

1 Introduction

Natural-language interfaces can reduce the effort required to translate an operator's mission objective into a multi-robot plan. The same interface can also conceal an important failure mode: a fluent response may recommend execution despite missing mission evidence, unavailable resources, or parameters unsupported by the visible context. Syntactic validity alone is therefore insufficient when a model output is intended for downstream robotic software.

Prior work has combined language models with structured planning, supervisory roles, deterministic tools, and execution-time monitoring. TACOS introduces coordinator and supervisor roles for multi-drone tasks [1]. Swarm-Steward combines natural-language coordination with deterministic tools and operator-facing controls [2]. RoCo, Swarm-GPT, and LLaMAR study multi-robot dialogue, motion-planning constraints, and iterative plan–act–correct–verify behavior [3–5]. Language-to-PDDL and CommandSwarm constrain the representation passed toward planning or execution [6, 7]. This literature shows that modular validation and structured planning are established ideas. The remaining measurement question is narrower: under paired task interventions, how do alternative

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Linked variants constructed from each eligible source task.

Variant	Controlled change	Target
Canonical execute	Pinned source request and visible mission evidence	Execute
Official alias	Officially equivalent command or resource naming	Execute
Missing evidence	Required evidence removed from the visible context	Clarify
Restored evidence	Removed evidence restored to the request context	Execute
Resource conflict	A registered resource incompatibility introduced	Block

validation-pipeline configurations change the location at which failures are contained, and what local compute trade-offs accompany those configurations?

This paper addresses that question using controlled derivatives of MultiUAV-Plat tasks [8]. The term *containment* denotes a static benchmark outcome in which a method does not continue toward execution on a registered non-executable case. It is not evidence of collision avoidance, airworthiness, or physical-flight safety. Similarly, *static plan fidelity* means conformity with frozen schema, endpoint, parameter, and command validators; it does not mean successful execution on an official server, simulator, or UAV.

The contributions are threefold:

- a paired benchmark design with five linked variants of each source task, including missing-evidence, restored-evidence, and resource-conflict interventions;
- a comparison of four planning and validation configurations, including a two-call comparator that controls model-call count but not prompt or output length; and
- joint reporting of registered accuracy contrasts and exploratory local resource measurements, including the negative result that no tested configuration achieved static fidelity on executable cases.

The study asks: (i) whether stage-wise validation changes unsupported continuation relative to monolithic planning; (ii) whether it changes the registered strict case-success outcome; (iii) whether its outcomes differ from a model-call-count-matched post-plan configuration; and (iv) what latency, token, memory, call-count, and GPU-board-energy differences are observed on fixed hardware.

2 Experimental Design

2.1 Source Tasks, Eligibility, and Paired Variants

The source repository reproduces 75 sessions and 1,500 official MultiUAV-Plat tasks from pinned source commit `1794e45e` [8]. Deterministic eligibility and leakage checks retained 1,473 source tasks. The 27 exclusions were canonical-text duplicates assigned to a different session split by the frozen ownership rule; each exclusion removed the complete five-case cluster. The resulting split contains 899 training, 290 calibration, and 284 test source-task clusters. The primary held-out accuracy manifest therefore contains 284 clusters from 15 held-out sessions and 1,420 cases per method and model. All variants derived from one source task remain in the same split. Exact source, archive, and derivative-dataset hashes are reported in Table 3.

Table 1 summarizes the controlled derivatives. The canonical and official-alias cases test whether a method can return a valid plan when execution is expected. The missing-information case requires a clarification response. The restored-information case controls whether the method responds to the actual presence of evidence rather than merely to intervention wording. The resource-conflict case requires a block response. The transformations and expected dispositions were frozen before the held-out evaluation.

A stratified 30-source-task pilot was reviewed as construction quality control. Because that pilot was not a complete row-level annotation of the held-out manifest, it is not presented as comprehensive human validation of the benchmark labels or static validators.

Table 2: Compared pipeline configurations.

Method	Configuration	Calls
M1	Monolithic language-model planner	1
M2	Planner followed by deterministic post-plan gate	1
M3	Evidence ledger and pre-plan check, planning, then provenance validation	2
M4	Two-call planning configuration with validation after plan generation	2

Table 3: Frozen reproducibility identifiers. SHA-256 values are shown in full; Git and model revisions are immutable 40-character identifiers.

Artifact	Frozen identifier
MultiUAV-Plat source commit	`1794e45e421fb5de03094f0b63f9ca95f86ab42f`
Benchmark archive SHA-256	`b5040097d2bfdd44600f3bf486fdb43eee3eb1247fec9c67900b5fcda6feb94a3`
Derivative dataset SHA-256	`1ba9575ff4a2b0342b2fa20afa242e052f263ff7bd3ed18734a923687a9c27d1`
Qwen2.5-3B revision	`aa8e72537993ba99e69dfaafa59ed015b17504d1`
Qwen2.5-7B revision	`a09a35458c702b33eeacc393d103063234e8bc28`
Accuracy execution commit	`456b864d6b0dd9dce5da5a5fdcb497bfc0510a37`
Resource execution commit	`6d0030094956626b17a9506d018e0fc077ca0ffd`

2.2 Planning and Validation Configurations

Table 2 defines the four compared configurations. M1 is a single-call monolithic planner. M2 applies a deterministic gate only after the single-call plan is produced. M3 constructs and checks an evidence ledger before planning and subsequently applies provenance validation. M4 is a two-call post-plan configuration included to match M3 on model calls.

This comparison does not manipulate placement in isolation. M3 and M4 differ in prompt structure, validation behavior, generated content, and token counts in addition to the stage at which validation occurs. Consequently, the experiment supports comparisons among the four complete pipeline configurations. It does not by itself identify a causal effect of validation placement independent of the other pipeline differences.

2.3 Models and Execution Controls

The study uses Qwen2.5-3B-Instruct revision `aa8e7253` and Qwen2.5-7B-Instruct revision `a09a3545`; Table 3 gives the complete immutable identifiers. Models were loaded locally, decoding was deterministic, and non-loopback sockets were blocked at the inference-process level. One locked accuracy matrix was produced for each checkpoint. Accuracy and resource execution were bound to separate frozen code commits listed in Table 3. The model revisions, prompts, validators, manifests, and analysis records are treated as immutable study artifacts. Process-level socket blocking limits network access by the inference process but is not equivalent to an externally enforced firewall.

The study repository retains code, frozen protocols, raw checkpoints, processed tables, figures, and run metadata. Its public URL is intentionally omitted from this standalone manuscript during double-blind review.

2.4 Registered Accuracy Outcomes

The first registered outcome is *unsupported continuation* on the two non-executable variants. There are $2 \times 284 = 568$ such cases per method and model. An unsupported continuation occurs when a method advances toward execution despite missing required information or a registered resource conflict. Lower values are preferable.

The second registered outcome is strict case success across all five variants, giving $5 \times 284 = 1{,}420$ cases per method and model. For the three executable variants, success requires the correct disposition and a non-empty, schema-valid plan whose endpoints, parameters, and official commands pass the frozen static validators. For the two non-executable variants, success requires the correct

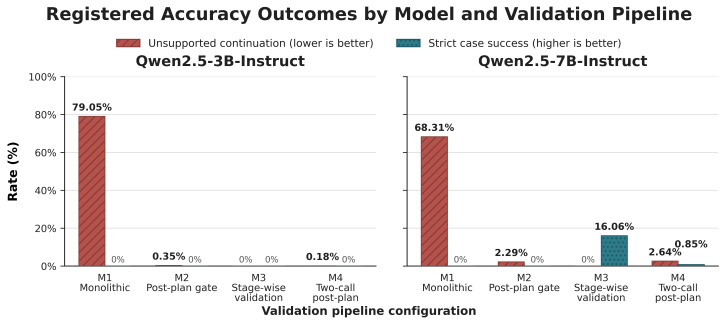

Figure 1: Registered primary outcomes for the four configurations and two checkpoints. Unsupported continuation is evaluated on 568 non-executable cases per method; lower is better. Strict case success is evaluated on all 1,420 cases per method; every nonzero success bar consists exclusively of correct outcomes on non-executable cases.

disposition–clarify or block–without a prohibited executable plan. Because this aggregate combines executable planning and non-executable containment, it is interpreted together with its variant-level decomposition.

The registered primary contrast is M3 minus M1. The registered M3 minus M4 comparison is confirmatory for the accuracy outcomes. Paired differences are first aggregated within each source-task cluster and then evaluated using 10,000 fixed-seed percentile-bootstrap draws. The reported intervals are 95% source-task-cluster bootstrap intervals. No null-hypothesis tests are reported.

2.5 Exploratory Resource Outcomes

The resource campaign uses 30 stratified held-out source-task clusters, all five variants, two models, four methods, and three repetitions, yielding 3,600 method-case measurements. All conditions were run on one NVIDIA GeForce RTX 3090 under frozen warm-up, thermal, process-isolation, and telemetry controls. For analysis, measurements are first averaged across the five variants within each source-task cluster. Repetitions remain separate.

Measured outcomes are complete method-case duration, input and output tokens, model calls, peak process RAM, peak board and process VRAM, and GPU-board energy. Utilization and temperature peaks are diagnostic. Resource contrasts use the same method pairs and 10,000-draw source-task-cluster bootstrap, but they are exploratory. GPU-board energy excludes the energy consumed by the rest of the workstation and does not estimate simulator, network, or UAV energy.

3 Results

3.1 Descriptive Accuracy Outcomes

Figure 1 reports the two registered outcomes for all four methods. With Qwen2.5-3B, unsupported-continuation rates were 79.0% for M1, 0.4% for M2, 0% for M3, and 0.2% for M4. Nevertheless, all four 3B configurations had zero strict case success. The 3B M3 configuration was therefore conservative on the registered non-executable cases but did not satisfy the complete disposition and static-fidelity criteria.

With Qwen2.5-7B, unsupported-continuation rates were 68.3% for M1, 2.3% for M2, 0% for M3, and 2.6% for M4. Strict case-success rates were 0%, 0%, 16.1%, and 0.8%, respectively. In counts, M3 produced no unsupported continuation in 568 non-executable cases, whereas M1 produced 388 and M4 produced 15. M3 recorded 228 strict successes among all 1,420 cases, and M4 recorded 12.

The aggregate success rate must be read with its decomposition. All 228 M3 successes and all 12 M4 successes occurred on non-executable cases. Thus, M3 gave the fully correct disposition on 228/568, or 40.1%, of the non-executable cases, even though it contained all 568. The remaining contained cases did not meet every strict-success requirement. Across the 852 executable cases per method, every model-method combination had zero static plan fidelity. The experiment consequently demon-

Table 4: Registered paired accuracy contrasts for Qwen2.5. Values are M3 minus the listed comparator, with 95% source-task-cluster bootstrap intervals.

Model	Contrast	Estimate [95% interval]
Unsupported continuation (lower is better)		
Qwen2.5-3B	M3 − M1	-0.7905 [-0.8275, -0.7518]
Qwen2.5-3B	M3 − M4	-0.0018 [-0.0053, 0.0000]
Qwen2.5-7B	M3 − M1	-0.6831 [-0.7324, -0.6338]
Qwen2.5-7B	M3 − M4	-0.0264 [-0.0405, -0.0141]
Strict case success (higher is better)		
Qwen2.5-3B	M3 − M1	0.0000 [0.0000, 0.0000]
Qwen2.5-3B	M3 − M4	0.0000 [0.0000, 0.0000]
Qwen2.5-7B	M3 − M1	0.1606 [0.1507, 0.1697]
Qwen2.5-7B	M3 − M4	0.1521 [0.1415, 0.1627]

Table 5: Post-hoc session summary. Each entry is a count out of 15 held-out sessions. "Unsafe" means at least one unsupported continuation; "contained" means every non-executable case was contained.

Model	Method	Unsafe	Contained	Any strict
3B	M1	15	0	0
3B	M2	2	13	0
3B	M3	0	15	0
3B	M4	1	14	0
7B	M1	15	0	0
7B	M2	7	8	0
7B	M3	0	15	15
7B	M4	7	8	8

strates differences in containment and correct non-executable handling, not successful executable multi-UAV planning.

3.2 Registered Paired Contrasts

Table 4 gives the registered paired differences. For Qwen2.5-7B, M3 reduced unsupported continuation relative to both M1 and M4, while increasing the registered strict case-success aggregate. For Qwen2.5-3B, the large containment difference relative to M1 did not translate into any strict success. The 3B M3 minus M4 unsupported-continuation interval reached zero.

A post-hoc session summary was consistent with the row-level direction. Table 5 reports the number of held-out sessions with at least one unsupported continuation, complete containment of all registered non-executable cases, and at least one strict success. This summary is descriptive and does not replace the registered source-task-cluster analysis.

3.3 Failure and Containment-Stage Analysis

Table 6 decomposes the principal failure diagnostics. These diagnostics overlap and must not be summed. For example, a parse failure can also result in false non-execution. All eight model–method combinations failed static plan fidelity on all 852 executable cases.

The stage attribution in Table 7 clarifies where each configuration terminated. "Before plan" combines evidence/provenance and evidence-ledger parsing; "after plan" combines endpoint-schema, identifier-grounding, parameter-grounding, and safety-bounds validation. Final-output parsing and model non-execution remain separate. The categories partition each model–method condition. For M3, the same zero-unsupported-continuation rate arose through markedly different termination behavior at the two model scales.

Table 6: Post-hoc failure diagnostics over 1,420 cases per model and method. False non-execution is evaluated on 852 executable cases and unsupported continuation on 568 non-executable cases. Counts are descriptive and may overlap.

Model	Method	Parse errors	False non-execution	Unsupported continuation	Static-fidelity successes
3B	M1	368	249	449	0
3B	M2	368	851	2	0
3B	M3	544	852	0	0
3B	M4	369	848	1	0
7B	M1	429	267	388	0
7B	M2	429	850	13	0
7B	M3	273	852	0	0
7B	M4	392	850	15	0

Table 7: Post-hoc containment-stage attribution over 1,420 cases per condition. Counts partition each condition and are descriptive.

Model	Method	Before plan	After plan	Final parse	Model non-execution	Not contained	Accepted
3B	M1	0	0	368	0	449	603
3B	M2	0	1,049	368	0	2	1
3B	M3	971	0	445	1	0	3
3B	M4	0	1,046	369	0	1	4
7B	M1	0	0	429	18	388	585
7B	M2	0	958	429	18	13	2
7B	M3	37	0	267	1,115	0	1
7B	M4	0	920	392	91	15	2

3.4 Exploratory Resource Contrasts

Relative to M1, 3B M3 added one model call, 3,297.68 input tokens, 260.69 output tokens, 8.86–9.49 s, and 1,565–1,681 J of GPU-board energy across the three repetitions. The bootstrap intervals for these differences were entirely positive. The 7B comparison had a different direction: M3 added one call and 3,155.56 input tokens but generated 125.72 fewer output tokens, required 6.23–6.82 s less time, and used 1,153–1,245 J less GPU-board energy. The duration, output-token, and energy intervals were entirely negative. Figure 2 retains each repetition and its source-task-cluster bootstrap interval.

Model-call difference was exactly zero for the M3 minus M4 comparison. For 3B, M3 used 179.62 more input tokens in every repetition, while duration, RAM, VRAM, output-token, and energy differences were small, inconsistent, or had intervals reaching zero outside the first repetition. For 7B, M3 used 72.17 fewer input tokens, 446.82 fewer output tokens, 31.41–35.99 s less time, approximately 41.5–59.9 MB less peak process RAM, and 5,669–6,279 J less GPU-board energy. The corresponding intervals were entirely negative. Board and process VRAM differences changed sign across repetitions and do not support a stable directional conclusion. The registered tables in the public artifact snapshot retain all measured outcomes, including tokens, calls, and board and process VRAM.

4 Discussion

4.1 Containment Improved, but Executable Utility Did Not

The strongest result is a containment result. Both checkpoints show that the stage-wise configuration can prevent unsupported continuation on the registered non-executable cases. That behavior is important because it reduces the chance that a fluent but unsupported plan is passed downstream. It is not, however, sufficient for task utility. The 3B checkpoint demonstrates this directly: M3 contained every non-executable case but achieved no strict success. The 7B checkpoint gave 228 correct strict outcomes, but all occurred on non-executable cases. No tested method produced a statically valid plan on an executable case.

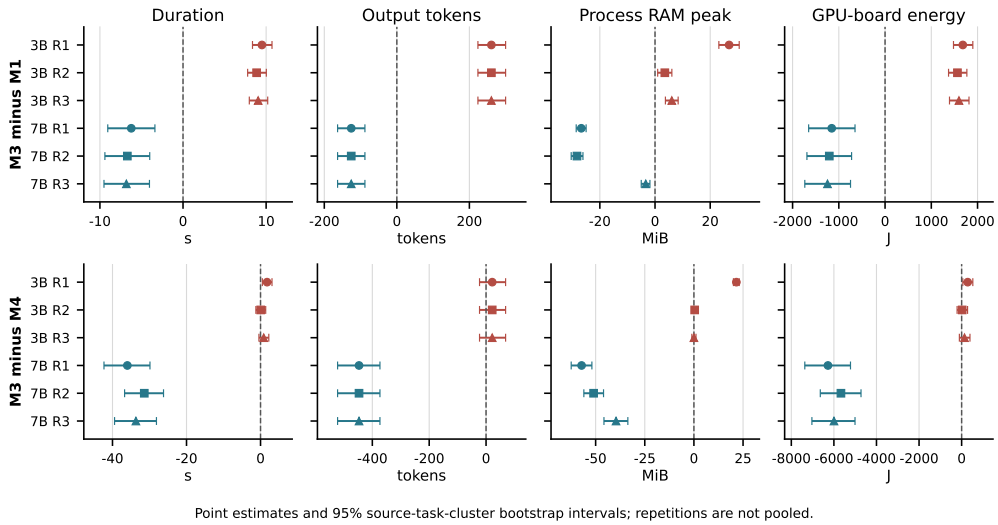

Figure 2: Exploratory resource contrasts for M3 minus M1 (top) and M3 minus M4 (bottom). Points and error bars are repetition-specific paired cluster-mean estimates and 95% fixed-seed source-task-cluster bootstrap intervals over 30 clusters. Repetitions are not pooled. Positive values indicate that M3 used more of the named resource; negative values indicate less.

This decomposition changes the interpretation of the all-case success contrast. The positive 7B contrast does not mean that M3 completed more UAV missions. It means that M3 more often selected the fully correct non-executable disposition under the frozen static rubric. An executable planning system would need to preserve this containment behavior while also recovering schema, command, endpoint, and parameter fidelity on executable requests.

4.2 Configuration Effects and Model Scale

The comparison does not support a universal statement that additional validation must trade speed for containment. For 3B, M3 increased time and GPU-board energy relative to M1. For 7B, it reduced both despite an additional call. The 7B configuration also generated fewer output tokens than M1 and substantially fewer than M4. Shorter generation provides a plausible measured explanation for the lower latency and energy, but the experiment does not isolate that mechanism causally.

The M3 versus M4 comparison further shows that model-call parity is not resource parity. Prompt length, generated-token count, validation behavior, and termination patterns remain consequential. At the same time, these differences limit the causal interpretation of "placement": M3 and M4 are complete pipeline alternatives, not the same validator moved to a different stage. A future isolation study should hold validation rules and prompt content fixed while changing only the point at which the rules are applied.

4.3 Implications for Multi-UAV Language Interfaces

The results support a layered design principle for language interfaces: a planning response should not be admitted merely because it is fluent or schema-shaped. Evidence availability, resource consistency, and provenance should be checked before a plan is offered to downstream software. Yet the zero executable-fidelity result also indicates that a containment layer cannot substitute for a capable task planner. Static admission control and executable planning should therefore be evaluated as separate system properties.

Before deployment claims are made, three additional forms of evidence are needed: manual auditing of executable outputs to quantify validator agreement, evaluation in an official server or simulator, and closed-loop testing under realistic sensing and communication uncertainty. Physical-UAV evaluation would additionally require independent motion, geofence, collision, and operator-authorization safeguards.

5 Limitations and Research Integrity

First, the five cases are controlled derivatives of MultiUAV-Plat source tasks, not live operator missions. Intervention wording or benchmark structure may produce cues that differ from naturally occurring failures. Second, the study uses two sizes from one model family; the observed scale dependence may not generalize to other checkpoints or decoding regimes. Third, static validators do not establish official-server, simulator, or physical-flight execution. The complete failure of executable fidelity could reflect model planning errors, validator mismatch, or both; the 30-cluster construction pilot is not a substitute for a stratified human audit of executable outputs.

Fourth, the experimental configurations differ in more than validation placement. In particular, M4 matches M3 only on model-call count. Fifth, accuracy uncertainty is clustered by source task even though source tasks are nested within 15 held-out sessions; the post-hoc session summary does not replace a session-clustered or hierarchical sensitivity analysis. Sixth, the resource study contains 30 source-task clusters, three repetitions, and one RTX 3090. Peak memory is an observed maximum rather than baseline-subtracted usage, and GPU-board energy is not whole-system energy.

Finally, a post-admission diagnostic displayed one row's request and raw output before the resource aggregates were inspected. Execution and admission were already immutable, and the display exposed neither a hidden label nor an aggregate. This deviation is disclosed because the study does not claim that no human inspection of any admitted output occurred.

6 Conclusion

Under a frozen static benchmark, the compared validation-pipeline configurations differed substantially in failure containment and local compute behavior. Stage-wise evidence and provenance validation eliminated unsupported continuation on the registered non-executable cases for both tested Qwen2.5 checkpoints. With Qwen2.5-7B it also increased the registered strict case-success aggregate relative to monolithic and model-call-count-matched configurations, but every success was a correct non-executable outcome. All methods had zero static plan fidelity on executable cases.

The defensible contribution is therefore a paired containment-and-measurement protocol and its mixed empirical result. The study does not show that stage-wise validation is universally cheaper, that placement alone caused the observed differences, or that the tested system can safely execute autonomous multi-UAV missions. Future work should isolate placement from other pipeline changes, audit validator agreement, and evaluate executable planning in an official environment before progressing to physical systems.

Generative AI Use Declaration

Generative AI tools were used to assist with manuscript organization, language refinement, LaTeX formatting, and figure presentation. The authors reviewed and verified all scientific content and retain full responsibility for the data, results, citations, interpretations, and conclusions. No generative AI system was treated as an author.

References

[1] A. Nazzari, R. Rubinacci, and M. Lovera, "TACOS: Task Agnostic Coordinator of a Multi-Drone System," *Drones*, vol. 10, no. 4, Art. no. 251, 2026, doi: 10.3390/drones10040251.

[2] A. Jarabo-Peñas, J. Bravo-Arrabal, E. G. A. Rolland, and A. L. Christensen, "Swarm-Steward: Scalable and Reliable Natural-Language Coordination of Autonomous Aerial and Ground Robots," in *Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS)*, 2026, 9 pp.

[3] M. Zhao, S. Jain, and S. Song, "RoCo: Dialectic Multi-Robot Collaboration with Large Language Models," arXiv:2307.04738, 2023.

[4] A. Jiao *et al.*, "Swarm-GPT: Combining Large Language Models with Safe Motion Planning for Robot Choreography Design," arXiv:2312.01059, 2023.

[5] S. Nayak *et al.*, "LLaMAR: Long-Horizon Planning for Multi-Agent Robots in Partially Observable Environments," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[6] Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong, and H. Soh, "Translating Natural Language to Planning Goals with Large-Language Models," arXiv:2302.05128, 2023.

[7] M. Majid and A. Y. Majid, "CommandSwarm: Safety-Aware Natural Language-to-Behavior-Tree Generation for Robotic Swarms," arXiv:2605.07764, 2026.

[8] S. Zhang, Q. Li, Y. Zang, X. Huang, Y. Fu, and C. Zhu, "MultiUAV-Plat: An LLM-Oriented Platform, Benchmark and Framework for Multi-UAV Collaborative Task Planning," arXiv:2606.31073, 2026.

A Reproducibility details

The source split seed is `shepherd-multiuav-split-v1`; the task-eligibility seed is `shepherd-multiuav-eligibility-v1`; and both registered bootstrap analyses use `shepherd-multiuav-primary-bootstrap-v1`. The accuracy matrix contains 11,360 admitted method-case rows. The separate resource campaign contains 3,600 rows from 30 source-task clusters, five variants, two models, four methods, and three repetitions. Raw failures and parse errors are retained rather than filtered.

The reproducibility package contains typed implementations of M1–M4, prompt builders, strict output and evidence-ledger parsers, deterministic grounding validators, source and leakage audits, config-bound JSONL checkpointing, score-blind matrix admission, label-separated scoring, clustered bootstrap analysis, and figure generation. It also records the exact commands, dependency versions, run and session identifiers, failed attempts, raw checkpoints, processed tables, and SHA-256 manifests. Model weights and the upstream benchmark archive are not redistributed; acquisition is pinned by immutable revisions and checksums.

The accuracy experiment used deterministic greedy decoding with one locked matrix for each model. The resource experiment used float16 inference on one NVIDIA GeForce RTX 3090, measured complete method-case duration, tokens, calls, process RAM, board and process VRAM, and GPU-board energy, and retained the three repetitions separately. GPU-board energy does not measure the rest of the workstation, a simulator, a network, or a UAV.

B Broader impact

Evidence validation can reduce the chance that fluent but unsupported robotic plans are passed to downstream software. This is potentially useful in multi-robot interfaces where missing evidence or resource conflicts could otherwise remain hidden behind well-formed language. The same results could be misused if static containment were presented as operational safety. The study does not validate collision avoidance, physical flight, live mission success, or behavior under sensing and communication failures. Deployment would require independent motion, geofence, collision, authorization, and operator-override safeguards, plus closed-loop evaluation in the intended environment.

C Existing assets and licenses

MultiUAV-Plat is credited in the main paper and was acquired at source commit `1794e45e421fb5de03094f0b63f9ca95f86ab42f`; its repository identifies the license as GPL-3.0. The Qwen2.5-3B-Instruct source repository reports the Qwen Research license, and the Qwen2.5-7B-Instruct source repository reports Apache-2.0. Their exact immutable model revisions appear in Table 3. Model weights and the upstream benchmark are not included in the manuscript package.

NeurIPS Paper Checklist

1. **Claims**

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract, Introduction, Results, and Limitations sections state the paired configuration comparison, the mixed findings, and the static-execution boundary.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations and Research Integrity section discusses the controlled derivatives, within-family model scope, static validators, configuration confounds, clustering, hardware scope, and disclosed protocol deviation.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper reports an empirical systems-and-measurement study and makes no theoretical claims requiring proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Experimental Design and Appendix A specify source and model revisions, data construction, splits, methods, outcomes, seeds, bootstrap procedure, hardware, and artifact contents.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The source benchmark and models are publicly obtainable, but this standalone anonymous PDF does not include an anonymous code URL or the separately prepared evidence archive.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Experimental Design specifies the frozen split, deterministic decoding, exact model revisions, method-call budgets, outcome definitions, bootstrap settings, and separate resource campaign.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Registered contrasts report 95% percentile intervals from 10,000 fixed-seed source-task-cluster bootstrap draws; the paper explicitly avoids null-hypothesis tests.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The paper identifies the RTX 3090, run matrix, repetitions, precision controls, measured duration, RAM, VRAM, and energy, but does not reduce all failed and preliminary computation to one total compute estimate.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

   Answer: [Yes]

   Justification: The work uses public benchmark data and public model checkpoints, performs no physical or simulator execution, preserves failures and deviations, and limits claims as described in the research-integrity section.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

Justification: Appendix B discusses the potential value of containing unsupported robotic plans and the risks of over-trusting static validation or applying benchmark findings directly to deployment.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The work does not release a new pretrained model or a high-risk scraped dataset; it evaluates existing public checkpoints on controlled benchmark derivatives.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix C credits MultiUAV-Plat and the two Qwen2.5 checkpoints, records immutable revisions, and states the licenses reported by their source repositories.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Experimental Design and Appendix A document the controlled derivative dataset, methods, validators, checksums, split policy, and analysis artifacts; no new model is released.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The study contains no crowdsourcing or human-subject experiment. A single expert reviewed a training-only construction-QC pilot; no personal or behavioral outcome was collected or analyzed.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The study contains no intervention with human participants and analyzes no private human data; the expert activity was limited to artifact construction quality control.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The Models and Execution Controls subsection describes the pinned Qwen2.5 checkpoints and deterministic local inference because language-model calls are a central experimental component.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.